

REPÚBLICA BOLIVARIANA DE VENEZUELA

MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA

UNIVERSIDAD NACIONAL EXPERIMENTAL DE TELECOMUNICACIONES E

INFORMÁTICA (UNETI)

PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

PROGRAMACION III MODULO 2

SECCIÓN 6Y7A-2026-1

**DESARROLLO DE UNA APLICACIÓN MÓVIL CON IONIC
FRAMEWORK Y ANGULAR: FORMULARIO DE CONTACTO
(PARTE 2)**

Participante:

Darwin Colmenares

C.I. V-19.064.945

Docente:

Carlos Márquez

Caracas, Venezuela
2026



ÍNDICE

Contenido

ÍNDICE.....	2
1. RESUMEN	3
2. INTRODUCCIÓN	3
3. DESARROLLO	5
3.1. Revisión del formulario base de la evaluación anterior	5
3.2. Implementación de validaciones personalizadas	8
3.3. Implementación de mensajes de error condicionales.....	11
3.4. Implementación de estilos dinámicos según validez.....	15
3.5. Implementación del método de envío con verificación	18
3.6. Pruebas funcionales del formulario	20
4. CONCLUSIÓN	25
5. REFERENCIAS BIBLIOGRÁFICAS	26



1. RESUMEN

El presente trabajo documenta el proceso de implementación de validaciones personalizadas en un formulario de contacto desarrollado con Ionic Framework y Angular. Partiendo del formulario base creado en la evaluación anterior, se desarrollaron validaciones programáticas para los campos de nombre, correo electrónico y mensaje, utilizando getters de TypeScript y expresiones regulares.

Se implementaron mensajes de error condicionales mediante directivas estructurales `*ngIf`, y estilos dinámicos que responden al estado de validez de cada campo. La metodología empleada

consistió en el análisis de requisitos de validación, la programación de métodos de verificación, la integración de retroalimentación visual en la interfaz de usuario, y la ejecución de pruebas funcionales para cada escenario posible. Los resultados evidencian un formulario robusto que impide el envío de datos incompletos o incorrectos, proporcionando retroalimentación inmediata al usuario mediante indicadores visuales y mensajes descriptivos. Se concluye que las validaciones en el lado del cliente constituyen una capa esencial de control de calidad en aplicaciones web y móviles.

Palabras clave: validaciones de formulario, Angular, Ionic Framework, TypeScript, expresiones regulares, retroalimentación de usuario.

2. INTRODUCCIÓN

En el desarrollo de aplicaciones web y móviles contemporáneas, los formularios constituyen el mecanismo fundamental de interacción entre el usuario y el sistema. La calidad de los datos ingresados a través de estos formularios determina en gran medida la confiabilidad de la información



procesada por la aplicación. En este contexto, las validaciones de entrada de datos representan una capa crítica de control que garantiza la integridad, consistencia y seguridad de la información.

Angular, como framework de desarrollo frontend, proporciona diversas estrategias para la validación de formularios, incluyendo validaciones sincrónicas, asíncronas, y personalizadas. Sin embargo, la implementación efectiva de estas validaciones requiere comprensión de conceptos como el two-way data binding, las propiedades computadas (getters), las directivas estructurales, y la gestión de estado de componentes.

El presente trabajo se enfoca en la implementación de validaciones personalizadas en el formulario de contacto desarrollado durante la evaluación anterior. El objetivo principal es transformar un

formulario básico de captura de datos en un componente robusto que valide la información en tiempo real, proporcione retroalimentación visual inmediata al usuario, y prevenga el envío de datos incompletos o mal formateados.

Esta actividad forma parte de la evaluación académica de la asignatura Programación III — Módulo 2, y tiene como propósito demostrar la capacidad del estudiante para programar lógica de negocio personalizada, manipular el estado de componentes, y diseñar interfaces de usuario con retroalimentación interactiva.

El documento está estructurado siguiendo las normas APA 7ma edición, y presenta un desarrollo detallado del código programado, acompañado de capturas de pantalla que evidencian cada etapa del proceso y los resultados de las pruebas funcionales.



3. DESARROLLO

3.1. Revisión del formulario base de la evaluación anterior

En la evaluación precedente se desarrolló una aplicación móvil con Ionic Framework y Angular que incluía una página de contacto con un formulario básico. Este formulario contaba con tres campos de entrada: nombre, correo electrónico y mensaje, vinculados al componente mediante la directiva `[(ngModel)]` que implementa el two-way data binding. Adicionalmente, se programó un método `enviarMensaje()` que mostraba una alerta con los datos ingresados.

Sin embargo, el formulario original carecía de mecanismos de validación. Cualquier usuario podía hacer clic en el botón "Enviar Mensaje" con los campos vacíos o con datos incorrectos, lo que generaba una experiencia de usuario deficiente y comprometía la calidad de la información capturada.

La Figura 1 muestra el código original del archivo `contacto.page.ts`, donde se aprecian las propiedades básicas y el método de envío sin validaciones.



```
darwinjcn Primera versión: App Ionic Angular con Side Menu, navegación y datos ...  
  
1 /**  
  
mi-app-ionic-evaluacion / src / app / pages / contacto / contacto.page.ts  
  
Code Blame 65 lines (61 loc) · 2.35 KB  
  
6 // y direccion en Caracas, Venezuela. Además, incluye un formulario  
7 * para que los visitantes puedan enviar un mensaje.  
8 */  
9 import { Component } from '@angular/core';  
10 import {  
11     IonContent, IonHeader, IonTitle, IonToolbar,  
12     IonList, IonItem, IonLabel, IonInput, IonTextarea, IonButton, IonIcon,  
13     IonMenuButton, IonButtons  
14 } from '@ionic/angular/standalone';  
15 import { FormsModule } from '@angular/forms';  
16 import { sendOutline, callOutline, locationOutline } from 'ionicons/icons';  
17 import { addIcons } from 'ionicons';  
18  
19 @Component({  
20     selector: 'app-contacto',  
21     templateUrl: './contacto.page.html',  
22     styleUrls: ['./contacto.page.scss'],  
23     standalone: true,  
24     imports: [  
25         IonContent, IonHeader, IonTitle, IonToolbar,  
26         IonList, IonItem, IonLabel, IonInput, IonTextarea, IonButton, IonIcon,  
27         IonMenuButton, IonButtons,  
28         FormsModule // Módulo necesario para el two-way data binding con ngModel  
29     ]  
30 })  
31 export class ContactoPage {  
32  
33     /**  
34     * Propiedades del componente para el formulario de contacto  
35     * Se utilizan con two-way data binding [(ngModel)] en el template HTML  
36     */  
37     nombre: string = ''; // Almacena el nombre ingresado por el usuario  
38     email: string = ''; // Almacena el correo electrónico ingresado  
39     mensaje: string = ''; // Almacena el mensaje ingresado por el usuario  
40
```

Figura 1. Código original del formulario de contacto sin validaciones.



La Figura 2 muestra la interfaz del formulario tal como se presentaba al usuario, sin indicadores de error ni retroalimentación.

The image shows a mobile application interface for a contact form. At the top, there is a purple header bar with a hamburger menu icon on the left and the word "Contacto" in white text. Below the header, the section "Información de Contacto" is displayed. It contains two items: a phone icon followed by "Teléfono: +58 426-5152896" and a location pin icon followed by "Dirección: Caracas, Distrito Capital, Venezuela". Below this, the section "Envíanos un mensaje" is shown. It includes three input fields labeled "Nombre", "Email", and "Mensaje". At the bottom of the form is a blue button with a right-pointing arrow icon and the text "ENVIAR MENSAJE".

Figura 2. Interfaz del formulario de contacto antes de las mejoras.



A partir de este estado inicial, se procedió a implementar un sistema de validaciones personalizadas que se detalla en las secciones siguientes.

3.2. Implementación de validaciones personalizadas

La primera etapa del desarrollo consistió en programar métodos de validación que verificaran el contenido de cada campo antes de permitir el envío del formulario. A diferencia de las validaciones automáticas que proporciona HTML5 (como el atributo `required`), estas validaciones fueron programadas manualmente en TypeScript, lo que permite control total sobre las reglas de negocio y los mensajes de error.

Se implementaron tres getters computados en el componente `contacto.page.ts`:

`nombreValido()`: Verifica que el campo nombre no esté vacío ni contenga solo espacios en blanco, utilizando el método `trim()` de JavaScript.

`emailValido()`: Valida el formato del correo electrónico mediante una expresión regular que verifica la presencia de caracteres alfanuméricos, el símbolo `@`, el dominio, y la extensión del dominio.

`mensajeValido()`: Comprueba que el mensaje tenga al menos 10 caracteres, garantizando que el usuario proporcione una descripción suficientemente detallada.

Adicionalmente, se programó la propiedad `formularioValido` que consolida el estado de todas las validaciones individuales, retornando `true` únicamente cuando los tres campos cumplen sus respectivas reglas.

La Figura 3 presenta el código programado de las validaciones personalizadas, donde se destacan los getters y la expresión regular para el correo electrónico.

```
TS contacto.page.ts M X
src > app > pages > contacto > TS contacto.page.ts > ContactoPage
38 export class ContactoPage {
57   intentoEnviar: boolean = false;
58
59   // =====
60   // VALIDACIONES PERSONALIZADAS (LO QUE PROGRAMO)
61   // =====
62
63   /**
64    * Valida que el campo nombre no esté vacío
65    * @returns true si el nombre es válido
66    */
67   get nombreValido(): boolean {
68     return this.nombre.trim().length > 0;
69   }
70
71   /**
72    * Valida el formato del correo electrónico
73    * Expresión regular que verifica: texto@texto.dominio
74    * @returns true si el email tiene formato correcto
75    */
76   get emailValido(): boolean {
77     const patronEmail = /^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
78     return patronEmail.test(this.email);
79   }
80
81   /**
82    * Valida que el mensaje tenga al menos 10 caracteres
83    * @returns true si el mensaje es suficientemente descriptivo
84    */
85   get mensajeValido(): boolean {
86     return this.mensaje.trim().length >= 10;
87   }
88
89   /**
90    * Verifica si todo el formulario es válido
91    * @returns true si todos los campos cumplen las validaciones
92    */
93   get formularioValido(): boolean {
94     return this.nombreValido && this.emailValido && this.mensajeValido;
95   }
```



Figura 3. Implementación de validaciones personalizadas mediante getters computados en TypeScript.

Es importante destacar que la expresión regular programada para el correo electrónico:

```
/^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/
```

Fue diseñada específicamente para esta aplicación. Esta expresión verifica que:

El correo comience con caracteres alfanuméricos, puntos, guiones bajos o guiones.

Contenga el símbolo @ obligatorio.

Incluya un dominio con caracteres válidos.

Termine con un punto seguido de al menos dos letras (extensión del dominio).

La Figura 4 muestra la propiedad intentoEnviar, un control de estado booleano que se utiliza para determinar cuándo mostrar los mensajes de error al usuario.

```
/**
 * Maneja el envío del formulario
 *
 * LO QUE SE PROGRAMA:
 * 1. Marca que se intentó enviar (para mostrar errores)
 * 2. Verifica si el formulario es válido
 * 3. Si es válido: muestra confirmación y limpia el formulario
 * 4. Si es inválido: no hace nada, los errores se muestran automáticamente
 */
enviarMensaje() {
  this.intentoEnviar = true;

  if (this.formularioValido) {
    alert("Mensaje enviado por: " + this.nombre + "\nEmail: " + this.email + "\nMensaje: " + this.mensaje);
    this.limpiarFormulario();
  }
}

/**
 * Restablece todos los campos del formulario a su estado inicial
 * Se ejecuta después de un envío exitoso
 */
```

Figura 4. Control de estado para la visualización de errores.

3.3. Implementación de mensajes de error condicionales

Una vez programadas las validaciones en TypeScript, se procedió a implementar la retroalimentación visual en la interfaz de usuario. El objetivo fue mostrar mensajes de error descriptivos únicamente cuando el usuario intenta enviar el formulario con datos inválidos, evitando así la visualización de errores prematuros mientras el usuario está escribiendo.

Para lograr este comportamiento, se utilizó la directiva estructural `*ngIf` de Angular en combinación con la propiedad `intentoEnviar`. La condición programada para cada mensaje de error es:

```
*ngIf="!nombreValido && intentoEnviar"
```



Esta expresión lógica garantiza que el mensaje se muestre solo si se cumplen dos condiciones simultáneamente: el campo es inválido Y el usuario ha intentado enviar el formulario.

Se programaron tres mensajes de error específicos:

- **"El nombre es obligatorio"** para el campo nombre vacío.
- **"Ingresa un correo electrónico válido"** para el campo email con formato incorrecto.
- **"El mensaje debe tener al menos 10 caracteres"** para el campo mensaje con longitud insuficiente.

La Figura 5 muestra el código HTML programado para los mensajes de error condicionales, utilizando el componente IonNote de Ionic con color "danger" para el indicador visual de error.

```
LO QUE SE PROGRAMA: validaciones visuales y mensajes de error
-->
<h3>Envíanos un mensaje</h3>

<!-- Campo Nombre -->
<ion-item [class.ion-invalid]="!nombreValido && intentoEnviar">
  <ion-label position="floating">Nombre</ion-label>
  <ion-input [(ngModel)]="nombre"></ion-input>
</ion-item>
<!--
Mensaje de error programado: solo aparece si es inválido Y se intentó
LO QUE SE PROGRAMA: condición *ngIf personalizada
-->
<ion-note color="danger" *ngIf="!nombreValido && intentoEnviar">
  El nombre es obligatorio
</ion-note>

<!-- Campo Email -->
<ion-item [class.ion-invalid]="!emailValido && intentoEnviar">
  <ion-label position="floating">Email</ion-label>
  <ion-input type="email" [(ngModel)]="email"></ion-input>
</ion-item>
<!--
Mensaje de error programado para email
LO QUE SE PROGRAMA: validación de formato con expresión regular
-->
<ion-note color="danger" *ngIf="!emailValido && intentoEnviar">
  Ingrese un correo electrónico válido
</ion-note>

<!-- Campo Mensaje -->
<ion-item [class.ion-invalid]="!mensajeValido && intentoEnviar">
  <ion-label position="floating">Mensaje</ion-label>
  <ion-textarea [(ngModel)]="mensaje"></ion-textarea>
</ion-item>
<!--
Mensaje de error programado para mensaje
LO QUE SE PROGRAMA: validación de longitud mínima (10 caracteres)
```

Figura 5. Implementación de mensajes de error condicionales mediante *ngIf.



La Figura 6 presenta la interfaz del formulario después de un intento de envío con campos vacíos, donde se aprecian los tres mensajes de error en color rojo debajo de cada campo.

The screenshot shows a mobile application interface for a contact form. At the top, there is a blue header with a hamburger menu icon and the title "Contacto". Below the header, the section "Información de Contacto" is displayed. It contains two rows: the first with a telephone icon and the text "Teléfono: +58 426-5152896", and the second with a location pin icon and the text "Dirección: Caracas, Distrito Capital, Venezuela". Below this, the section "Envíanos un mensaje" is shown. It contains three input fields: "Nombre", "Email", and "Mensaje". Each field has a red error message below it: "El nombre es obligatorio" for the name field, "Ingrese un correo electrónico válido" for the email field, and "El mensaje debe tener al menos 10 caracteres" for the message field. At the bottom, there is a grey button with a right-pointing arrow and the text "ENVIAR MENSAJE".

Figura 6. Visualización de errores tras intento de envío con campos vacíos.



3.4. Implementación de estilos dinámicos según validez

Para proporcionar una retroalimentación visual inmediata, se programaron estilos dinámicos que responden al estado de validez de cada campo y del formulario completo.

En el archivo HTML, se utilizó el binding de propiedades [property] de Angular para asignar dinámicamente la clase CSS ion-invalid a cada item cuando el campo es inválido y se ha intentado enviar:

```
[class.ion-invalid]="!nombreValido && intentoEnviar"
```

Esta programación permite que el borde del campo cambie de color automáticamente, proporcionando una señal visual clara sin requerir intervención manual del usuario.

Adicionalmente, se programó el cambio de color del botón de envío según el estado del formulario:

```
[color]="formularioValido ? 'success' : 'medium'"
```

Cuando todos los campos son válidos, el botón se muestra en color verde (success), indicando que el formulario está listo para enviar. Cuando existe algún error, el botón se muestra en color gris (medium), señalando visualmente que aún faltan correcciones.

La Figura 7 muestra el código programado para los estilos dinámicos en el template HTML.

```
LO QUE SE PROGRAMA: validaciones visuales y mensajes de error
-->
<h3>Envíanos un mensaje</h3>

<!-- Campo Nombre -->
<ion-item [class.ion-invalid]="!nombreValido && intentoEnviar">
  <ion-label position="floating">Nombre</ion-label>
  <ion-input [(ngModel)]="nombre"></ion-input>
</ion-item>
<!--
Mensaje de error programado: solo aparece si es inválido Y se intentó enviar
LO QUE SE PROGRAMA: condición *ngIf personalizada
-->
<ion-note color="danger" *ngIf="!nombreValido && intentoEnviar">
  El nombre es obligatorio
</ion-note>

<!-- Campo Email -->
<ion-item [class.ion-invalid]="!emailValido && intentoEnviar">
  <ion-label position="floating">Email</ion-label>
  <ion-input type="email" [(ngModel)]="email"></ion-input>
</ion-item>
<!--
Mensaje de error programado para email
LO QUE SE PROGRAMA: validación de formato con expresión regular
-->
<ion-note color="danger" *ngIf="!emailValido && intentoEnviar">
  Ingrese un correo electrónico válido
</ion-note>

<!-- Campo Mensaje -->
<ion-item [class.ion-invalid]="!mensajeValido && intentoEnviar">
  <ion-label position="floating">Mensaje</ion-label>
  <ion-textarea [(ngModel)]="mensaje"></ion-textarea>
</ion-item>
<!--
Mensaje de error programado para mensaje
LO QUE SE PROGRAMA: validación de longitud mínima (10 caracteres)
-->
```

Figura 7. Implementación de estilos dinámicos mediante binding de propiedades.

La Figura 8 presenta el formulario con datos válidos, donde se aprecia el botón en color verde indicando que el formulario está listo para el envío.

```
app > pages > contacto > contacto.pugentini > ion-content ion-padding
<ion-content class="ion-padding">
  -->
  <ion-note color="danger" *ngIf="!emailValido && intentoEnviar">
    | Ingrese un correo electrónico válido
  </ion-note>

  <!-- Campo Mensaje -->
  <ion-item [class.ion-invalid]="!mensajeValido && intentoEnviar">
    <ion-label position="floating">Mensaje</ion-label>
    <ion-textarea [(ngModel)]="mensaje"></ion-textarea>
  </ion-item>

  <!--
  | Mensaje de error programado para mensaje
  | LO QUE SE PROGRAMA: validación de longitud mínima (10 caracteres)
  -->
  <ion-note color="danger" *ngIf="!mensajeValido && intentoEnviar">
    | El mensaje debe tener al menos 10 caracteres
  </ion-note>

  <!--
  | Botón de envío programado:
  | - color cambia según validez (success si válido, medium si inválido)
  | - al hacer clic ejecuta enviarMensaje() que programé
  -->
  <ion-button
    expand="block"
    [color]="formularioValido ? 'success' : 'medium'"
    (click)="enviarMensaje()"
    class="ion-margin-top">
    <ion-icon name="send-outline" slot="start"></ion-icon>
    Enviar Mensaje
  </ion-button>
</ion-content>
```

Figura 8. Indicador visual de formulario válido (botón verde).



3.5. Implementación del método de envío con verificación

El método `enviarMensaje()` fue reprogramado para incluir una verificación previa del estado del formulario. A diferencia de la versión anterior, que mostraba la alerta incondicionalmente, la nueva implementación sigue la siguiente lógica programada:

- Establecer `intentoEnviar = true`, lo que activa la visualización de errores.
- Evaluar la propiedad `formularioValido`.
- Si es `true`: mostrar alerta de confirmación y ejecutar `limpiarFormulario()`.
- Si es `false`: no realizar ninguna acción, permitiendo que los mensajes de error guíen al usuario.

Adicionalmente, se programó el método `limpiarFormulario()` que restablece todas las propiedades a su estado inicial:

```
this.nombre = "";
```

```
this.email = "";
```

```
this.mensaje = "";
```

```
this.intentoEnviar = false;
```

Este método garantiza que el formulario quede listo para una nueva interacción después de un envío exitoso.

La Figura 9 muestra el código programado del método `enviarMensaje()` con la estructura condicional de verificación.

```

    */
    enviarMensaje() {
        this.intentoEnviar = true;

        if (this.formularioValido) {
            alert("Mensaje enviado por: " + this.nombre + "\nEmail: " + this.email + "\nMensaje: " + this.mensaje);
            this.limpiarFormulario();
        }
    }

    /**
     * Restablece todos los campos del formulario a su estado inicial
     * Se ejecuta después de un envío exitoso
     */
    limpiarFormulario() {
        this.nombre = '';
        this.email = '';
        this.mensaje = '';
        this.intentoEnviar = false;
    }
}

```

Figura 9. Método de envío con verificación de validez previa.

La Figura 10 presenta la alerta de confirmación que se muestra cuando el usuario envía el formulario con datos válidos

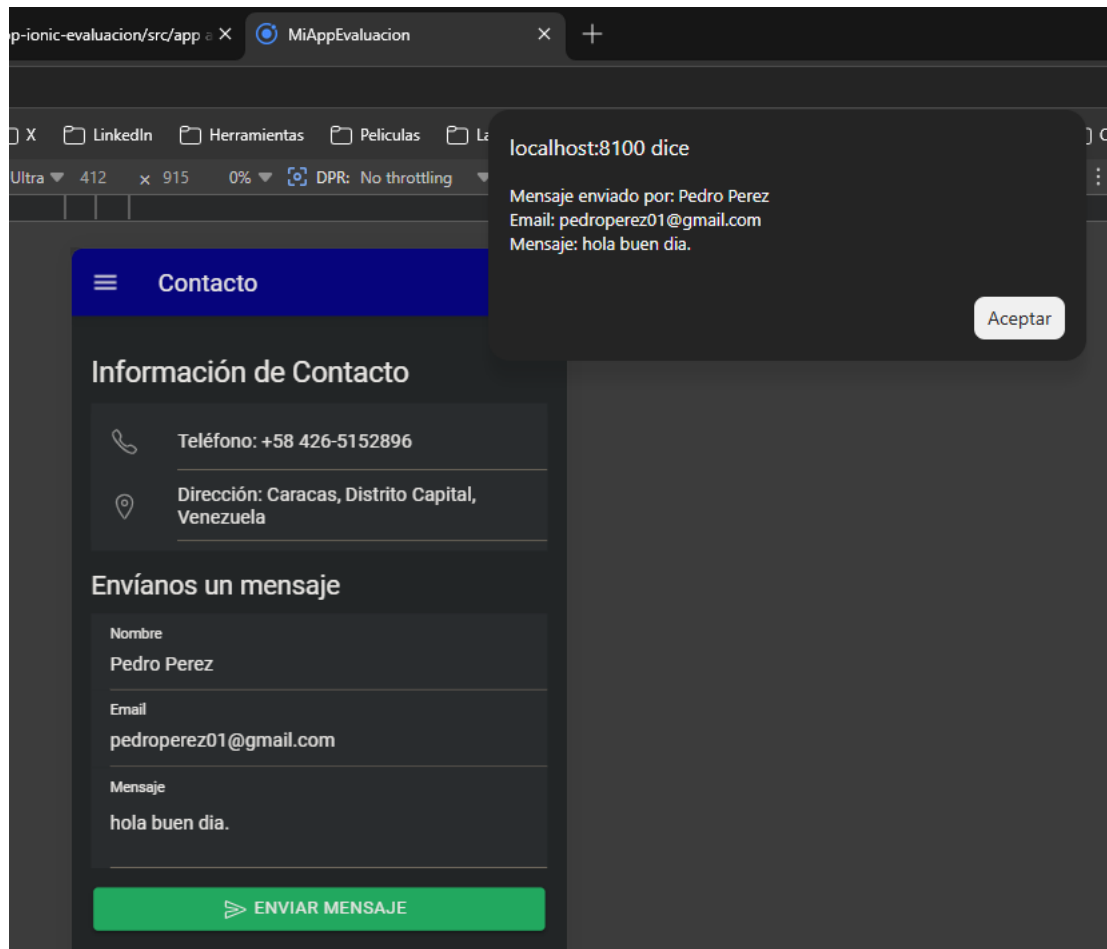


Figura 10. Confirmación de envío exitoso del formulario.

3.6. Pruebas funcionales del formulario

Para verificar el correcto funcionamiento del sistema de validaciones implementado, se ejecutaron pruebas funcionales que cubren los escenarios más comunes de uso:

Prueba 1: Intento de envío con todos los campos vacíos



Se hizo clic en el botón "Enviar Mensaje" sin ingresar datos en ningún campo. El resultado esperado fue la visualización de los tres mensajes de error. La Figura 11 confirma que el comportamiento programado se ejecutó correctamente.

The screenshot shows a mobile application interface for a contact. At the top is a blue header with a hamburger menu icon and the word "Contacto". Below this is a section titled "Información de Contacto" containing two items: a phone number "+58 426-5152896" and an address "Caracas, Distrito Capital, Venezuela". The main section is titled "Envíanos un mensaje" and contains three input fields: "Nombre", "Email", and "Mensaje". Each field has a red error message below it: "El nombre es obligatorio", "Ingrese un correo electrónico válido", and "El mensaje debe tener al menos 10 caracteres". At the bottom is a large button with a right-pointing arrow and the text "ENVIAR MENSAJE".

Figura 11. Prueba de envío con campos vacíos



Prueba 2: Validación de formato de correo electrónico

Se ingresó "correo-invalido" (sin @ ni dominio) en el campo email y se intentó enviar. La Figura 12 muestra el mensaje de error específico para formato inválido.

The screenshot shows a mobile application interface for a contact page. At the top, there is a blue header with a menu icon and the title "Contacto". Below this, the section "Información de Contacto" displays the contact's phone number as "+58 426-5152896" and their address as "Caracas, Distrito Capital, Venezuela". The "Envíanos un mensaje" section contains input fields for "Nombre" (filled with "Maria Medina") and "Email" (filled with "maria.medina&gmail.com"). A red error message, "Ingrese un correo electrónico válido", is displayed below the email field. There is also a "Mensaje" field containing the text "*assa", with a red error message below it stating "El mensaje debe tener al menos 10 caracteres". At the bottom, a grey button with a right-pointing arrow and the text "ENVIAR MENSAJE" is visible.

Figura 12. Prueba de validación de formato de email.



Prueba 3: Validación de longitud mínima del mensaje

Se ingresó "Hola" (4 caracteres) en el campo mensaje. La Figura 13 evidencia que el sistema detectó la longitud insuficiente y mostró el error correspondiente.

The screenshot shows a mobile application interface for a contact. At the top, there is a blue header with a hamburger menu icon and the word "Contacto". Below this is a section titled "Información de Contacto" which contains two rows: "Teléfono: +58 426-5152896" and "Dirección: Caracas, Distrito Capital, Venezuela". Below this section is another titled "Envíanos un mensaje". It contains three input fields: "Nombre" with the value "Henrique Gonzalez", "Email" with the value "henriqueg01@gmail.com", and "Mensaje" with the value "Hola". Below the "Mensaje" field, there is a red error message: "El mensaje debe tener al menos 10 caracteres". At the bottom of the form is a grey button with a right-pointing arrow and the text "ENVIAR MENSAJE".

Figura 13. Prueba de validación de longitud mínima.



Prueba 4: Formulario completado correctamente

Se ingresaron datos válidos en todos los campos: nombre "Natalia Jimenez", email "Natalia.jimenez@gmail.com", y mensaje "Este es un mensaje de prueba para la evaluación". La Figura 14 muestra el formulario con todos los indicadores positivos y el botón en verde

The screenshot shows a web browser window with the address bar displaying 'pp-ionic-evaluacion/src/app' and the page title 'MiAppEvaluacion'. The browser's developer tools are open, showing a console message from 'localhost:8100' stating: 'Mensaje enviado por: Natalia Jimenez', 'Email: Natalia.jimenez@gmail.com', and 'Mensaje: Este es un mensaje de prueba para la evaluación.' Below the console, a toast notification with an 'Aceptar' button is visible. The main content area features a 'Contacto' section with a blue header. Under 'Información de Contacto', there is a phone number '+58 426-5152896' and an address 'Caracas, Distrito Capital, Venezuela'. The 'Envíanos un mensaje' section contains three input fields: 'Nombre' (filled with 'Natalia Jimenez'), 'Email' (filled with 'Natalia.jimenez@gmail.com'), and 'Mensaje' (filled with 'Este es un mensaje de prueba para la evaluación.'). A green button labeled 'ENVIAR MENSAJE' is positioned at the bottom of the form.

Figura 14. Prueba de formulario válido.



4. CONCLUSIÓN

La implementación de validaciones personalizadas en el formulario de contacto permitió transformar un componente básico de captura de datos en una interfaz robusta con retroalimentación interactiva. Mediante la programación de getters computados en TypeScript, se establecieron reglas de negocio específicas para cada campo: obligatoriedad del nombre, formato correcto del correo electrónico mediante expresiones regulares, y longitud mínima del mensaje.

La integración de mensajes de error condicionales mediante la directiva `*ngIf`, controlados por la propiedad `intentoEnviar`, garantizó que la retroalimentación al usuario apareciera en el momento oportuno: después de un intento de envío, no durante la escritura. Esta decisión de diseño mejora significativamente la experiencia de usuario al evitar mensajes de error prematuros.

Los estilos dinámicos programados mediante binding de propiedades `[class]` y `[color]` proporcionaron indicadores visuales inmediatos que complementan los mensajes textuales. El cambio de color del botón de envío según la validez del formulario constituye una señal intuitiva que guía al usuario hacia la corrección de errores.

Durante el proceso se enfrentaron desafíos técnicos como la sincronización entre el estado del componente TypeScript y la renderización condicional en el template HTML. La resolución de estos problemas requirió comprensión del ciclo de detección de cambios de Angular y del flujo de datos unidireccional en componentes standalone.

El proyecto cumple con los requisitos establecidos para la evaluación: el formulario de contacto ahora incluye validaciones programadas manualmente,



mensajes de error descriptivos, estilos dinámicos, y un método de envío con verificación previa. Este trabajo fortalece las competencias de programación de lógica de negocio y diseño de interfaces interactivas, sentando bases para el desarrollo de formularios más complejos con validaciones asíncronas e integración con servicios backend.

5. REFERENCIAS BIBLIOGRÁFICAS

Angular. (2026). **Angular documentation**. <https://angular.dev>

Colmenares, D. (2026). **miAppEvaluacion: Aplicación móvil desarrollada con Ionic Framework y Angular** [Repositorio GitHub]. Disponible: <https://github.com/darwinjcn/mi-app-ionic-app>

Ionic. (2026). **Ionic Framework documentation**. Disponible: <https://ionicframework.com/docs>

Ionic. (2026). **Ionicons: The premium icon pack for Ionic**. Disponible: <https://ionicons.com>

Mozilla Developer Network. (2026). Regular expressions. Disponible: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions

Node.js Foundation. (2026). **Node.js documentation**. Disponible: <https://nodejs.org/en/docs/>